Listing 5.4 An example of a simple GraphQL API that orchestrates calls to two RESTful APIs to return weather data for the requested city.

```
import graphene, requests
from flask import Flask, request, jsonify

class WeatherData(graphene.ObjectType): #1
  time = graphene.List(graphene.String)
  rain = graphene.List(graphene.Float)
  surface_temperature = graphene.List(graphene.Float)

class Query(graphene.ObjectType): #2
  weather = graphene.Field(WeatherData, city=graphene.String(required
=True))

  def resolve_weather(self, info, city):
    geo_data = requests.get( #3
      f"https://geocoding-api.open-meteo.com/v1/"
      f"search?name={city}&count=1"
    ).json()

    if not geo_data.get("results"):
      raise Exception(f"City '{city}' not found")

    lat, lon = (geo_data["results"][0][k] for k in ["latitude", "long
itude"])

    weather_data = requests.get( #4
      f"https://api.open-meteo.com/v1/"
      f"forecast?latitude={lat}&longitude={lon}"
      f"&hourly=rain,surface_temperature&models=ecmwf_ifs025"
      f"&forecast_days=7"
    ).json()

    return WeatherData(**weather_data["hourly"])

schema = graphene.Schema(query=Query)
app = Flask(__name__)

@app.route("/graphql", methods=["POST"])
def graphql_server():
  try:
    result = schema.execute(request.get_json().get("query"))
    return jsonify(result.data)
```